

- 1) Este código gestiona una hacienda donde se manejan animales, donde se coordinan bovinos donde se les clasifica por edad, como novillo, ternero, o cebón, a las reses; Se les coordina por medio de potreros (donde debe haber un máximo de 150 reses), también se coordinan las vacunas y la venta de los animales. Respecto a la administración de reses este software maneja también lo que viene siendo carnet de vacunas de cada res, lleva cuentas de la edad de cada animal (en meses) para poder clasificarlo y lleva cuentas de su peso (en kilogramos) para clasificar si está en desnutrición o no; El software maneja reglas para registrar y no permitir que los valores se pasen de un rango, sino manda una excepcion/alerta. Respecto a la venta de reses, simplemente se maneja un método abstracto para venderlas. Respecto a la vacunación de reses, lleva un conteo de cuantas reses están bacterianas o están vivas y maneja unas reglas con un máximo de vacunas para cada tipo de res.
- 2) La clase potrero haría difícil un cambio porque utiliza muchas excepciones y utiliza switches que podrían manejarse de otra manera  
La clase validación, se termina validando muchas veces y muchas clases terminan heredando de la misma haciendo todo redundante  
La clase hacienda, que termina haciendo como de todo, tiene muchos métodos que hacen cosas relativamente distintas, pienso que es una mejor idea de diseño separar entre más clases o interfaces
- 3) Le preguntaría al programador original: ¿Por qué utilizas tantas validaciones? ¿No termina siendo redundante?  
¿Por qué generaste hacienda de manera tan monolítica? ¿No hay mejores manejos?  
¿Por qué utilizas crear vacunas para las vacas vivas 2 veces? ¿No es más fácil manejarlo de otra manera?  
¿Por qué utilizas tantos switches?